; Extending the first hps (homogeneous parallel server) queuing model
; Kanageshwaran Dhakshinamoorthy
; CSC148 student, Fall 2025
; This model extends the in-class FIRST hps (Module P2): each arriving cj requests a random 1–6 servers
; using transaction parameters (no SAVEVALUE), compares two service scenarios (load-free vs. load-dependent),
; and creates histograms of customer residence time (w_residence) and free servers at service start (free_srv_hist).
; Time unit (t.u.) = 1 (arbitrary)
; Exact START command used: START 25000
; Model termination condition: simulation ends when the 25,000th customer transaction completes; each
; TERMINATE 1 in cj reduces tc by 1; the initializer transaction init_hps TERMINATE does not decrement tc.
; Additional run details: Ai ~ Exponential(mean = 1.0 t.u.); Service ~ Exponential(mean = 5.0 t.u.);
; load-dependent service adds sc = 0.12 * (r$hpsName)^(2.19) / nps (or cubic approximation if ^2.19 unsupported).
; hps creation/initialization: hpsName STORAGE <nps>; init_hps GENERATE 0,,0,1,1 then ENTER hpsName,<nps> at t=0.
; Server use protocol: TEST GE s$hpsName,p$p_srvReq → LEAVE hpsName,p$p_srvReq → service → ENTER hpsName,p$p_srvReq.
; Seeds fixed across all runs per assignment: RN1 = 123456, RN2 = 789123.

==========

First HandIn part – Completed Table 1 and Table 2 (can cut & paste to get table layouts).
Table columns LEGEND – max(LQ)=(max number of cj that waited for service); avg(W)=avg(cj residence duration)

1A) For Table 1, Make runs, starting with nps = 16, and doubling nps on successive runs until a run occurs satisfying
□□□□□^ < 0.3 and max(LQ) <= 1 ***(this is the LoS metric)*** **In the row for each run's results fill in fourth through last columns.**

Table 1 – Runs with load free scenario

Ai mean (fixed)	Service duration mean 4.9 (fixed)	nps value of this hps	True □□ stat value, NOT .rpt UTIL. (as a fraction)	max(LQ)	avg(W)	RN1 seed	RN2 seed
1.0	4.9	16	0.996938	1526	860.270	123456	789123
1.0	4.9	32	0.540688	10	5.158	123456	789123

1.0	4.9	64	0.270547	1	5.008	123456	789123
1.0	4.9	128	0.135273	1	5.008	123456	789123

==========

1B) For Table 2, Make runs starting, with nps = 32, and doubling nps on successive runs until a run occurs satisfying

□□□□□^ < 0.3 AND max(LQ) <= 1. Fill in Table 2 as done for Table 1.

Note - The nps 32 run in Table 2 can take 20 or more seconds, depending on your machine's speed.

Table 2 – Runs with load dependent scenario

Ai mean □□□□□ fixed)	service duration mean 4.9 fixed	nps value of this hps	True □□ stat value , NOT .rpt UTIL. (as a fraction)	max(LQ)	avg(W)	RN1 seed	RN2 seed
1.0	4.9	32	0.998375	4881	3198.312	123456	789123
1.0	4.9	64	0.997188	2914	1739.189	123456	789123
1.0	4.9	128	0.184031	1	6.736	123456	789123
1.0	4.9	256	0.075711	1	5.572	123456	789123

==========

Second HandIn part - For the "load dependent" scenario and ONLY for nps = 64, provide all of the following:

2) The part of the .rpt file containing QUEUE and STORAGE sections only – just those 4 or 5 or report lines;

(DO NOT submit that entire Report file, since it might be several pages)

```
QUEUE            MAX CONT. ENTRY ENTRY(0) AVE.CONT. AVE.TIME   AVE.(-0)
RETRY
 Q_RESIDENCE     2933    0  25000     0  1457.091   1739.189   1739.189   0
 Q_WAIT          2914    0  25000   134  1438.760   1717.308   1726.562   0
 Q_SERVICE         27    0  25000     0    18.332     21.881     21.881   0


STORAGE          CAP. REM. MIN. MAX.  ENTRIES AVL.  AVE.C. UTIL. RETRY
DELAY
 HPSNAME          64    0   0    64    87090   1    0.180  0.003    0    0
```

==========

Third HandIn part

3) The histogram of (the number of free servers as each cj starts its service)

The start of service point in the source code is: immediately after the LEAVE block that cj used to acquire all servers.

